The Open Control Architecture

Jeff Berryman, *AES Member*
ja.berryman@us.bosch.com

Bosch Communications Systems, Burnsville, Minnesota, USA

The Open Control Architecture (OCA) is a detailed foundation for the definition of media system control protocols for various network types. It is intended for use in professional media networks of 2 to 10,000 nodes, including mission-critical networks that may be involved in life-safety systems. An evolution of AES-24, an AES control protocol architecture developed in the 1990's, OCA is designed to be an expandable public standard with a long service life.

0. INTRODUCTION

This report describes the Open Control Architecture (OCA), a control protocol architecture for media networks in the professional realm.

By "protocol architecture", we mean a system of data elements and related semantics which can be implemented by various communication protocols, depending on the network media involved.

By "media network", we mean a digital network which supports:

1. Synchronous transport of zero to many channels of audio and/or video; and
2. Remote control and monitoring of zero to many devices.

1. PURPOSE OF OCA

The purpose of OCA is to provide a basis for defining reliable, flexible, and scalable control protocols for professional media networks.

OCA is for system control and monitoring only, and may be integrated with any streaming program transport protocol scheme, as long as the underlying digital network offers a suitable form of connectivity for OCA-compliant control and monitoring traffic.

OCA is intended to be useful for both audio and video control and monitoring. However, the current version does not yet include video-specific elements.

OCA is distinguished from other current media system control designs in that it aspires to support the complete range of professional media network applications, ranging from local networks in studios, clubs, and conference rooms to large, mission-critical networks with long installed lifetimes and possibly very large physical extents. Scope includes both installed and touring systems, and applications in which the two are interconnected.

2. HISTORY

2.1. Acknowledgements

2.1.1. Development Team

The work described here was done by a team of technologiests from members of the OCA Alliance (2.2, below), as follows:

Jeff Berryman (the author)..... Bosch Communications
Gerald van Beuningen .....Duran Audio
Klas Dalbjörn..... Lab Gruppen
Hiroshi Hamamatsu ..... Yamaha
Morten Lave ..... TC Group
Nathan O'Neill ..... LOUD Technologies
Martin Renz ..... d&b audiotechnik
Marc Smaak..... Bosch Communications
Daisuke Takahashi..... Yamaha
Stephan van Tienen ..... Bosch Communications
Bob Tudor..... Presonus
Ethan Wetzell ..... Bosch Communications

2.1.2. AES-24

Many of OCA's core concepts are based on the work done by the AES standards subcommittee SC-10 for the AES-24 project. Although AES-24 did not ultimately become a standard, the work was published [2,3]. and has formed a conceptual basis for several commercial efforts since.

To the best of the authors' knowledge, OCA is the first AES-24 legacy effort to aim for open public standardization.

2.2. The OCA Alliance

The OCA Specification is being developed by the OCA Alliance. Founded in 2011 as an informal collaboration of nine companies, the Alliance is now a trade association whose mission is to develop, promote, and support OCA as an open public standard for control and monitoring of professional media networks.

2.3. Status of Standardization

In October of 2012, OCA was accepted into the AES standards process. Its standardization is the object of AES project X210, administered by AES Standards Subcommittee SC-02-12.

Future work will add video-specific elements to OCA. The video aspects of OCA will be outside the scope of AES; an additional standards development collaborator will be sought to address these.

The ultimate goal is to have OCA audio and video standards combined into a single international standard, published by an organization such as ISO or IEC.

2.4. Status of the Specification

The current proposed Specification version is 1.1. It is available for free download from the Alliance's website, www.oca-alliance.org. The Specification is described as "proposed" because the final version(s) will be published formally by the AES.

It is expected that the final versions will differ from the proposed version. Users of the proposed Specification should assume that their implementations will require updating to comply with the final version.

3. REQUIREMENTS

Functional requirements for professional media networks have been described elsewhere [4,18]. From these descriptions, the requirements relevant to OCA are paraphrased as follows ("PMN" means "Professional Media Network"):

3.1. **Standard Network Infrastructure Compatibility.** PMNs must be able to run over standard infrastructure, e.g. TCP/IP Ethernet.

3.2. **Existing Network Infrastructure Compatibility.** PMNs must be compatible with existing network infrastructures and management practices.

3.3. **Computing Platform Independence.** PMNs must be implementable in various computer and embedded environments, and the implementations on those diverse platforms must interoperate transparently.

3.4. **Node Count.** PMNs must support node counts up to at least 10,000.

3.5. **Diameter.** A network's "diameter" is the physical length of cable (or, for wireless networks, distance in air) between its two most widely-separated nodes. PMNs must support diameters range from a few meters to thousands of kilometers.

3.6. **Public Network Bridging**. It must be possible to interconnect separate sections of a PMN using public networks (e.g. the Internet) in a manner that provides sufficient security and performance.

3.7. **Reliability.** At all times, PMN controllers must have accurate knowledge of the media network's device population. Controllers must be reliably and quickly informed of device state changes, and must have positive acknowledgement of all commands sent to devices.

3.8. **Performance.** PMN control and monitoring performance requirements are variable, but it must be possible to obtain acceptably fast control performance from relatively simple devices.

3.9. **Evolvability.** It must be possible to add new devices to a PMN without compatibility problems, and it must be possible to upgrade to new versions of OCA without impacting legacy equipment.

3.10. **Support for Proprietary Features.** PMNs must integrate vendor-specific features cleanly, without loss of interoperability with other standard devices, and in ways that keep trade secrets from being revealed.

3.11. **Range of Control Capabilities.** PMNs must be able to control and monitor all functions of devices over a wide range of complexities. Reconfigurable signal processors must be fully supported.

3.12. **Multiple Controller Support.** PMNs must support multiple system controllers, with each controller performing some or all of the overall control function.

3.13. **Redundancy.** PMN standards must allow for redundant implementations, with multiple network paths and duplicated controllers.

3.14. Ad-hoc Interconnection. It must be possible to interconnect PMNs without catastrophic consequences.

3.15. **Security.** The PMN standard must provide options for efficient authentication and secure encryption of both media program and system control traffic. These options must use standard algorithms that are free from international restriction.

3.16. **Maintenance Support.** PMNs should support reliable and secure remote firmware/software upgrading.

3.17. **Multivendor Interoperability.** It must be possible to construct PMNs that integrate equipment from multiple manufacturers.

3.18. **Safety Standards Compliance.** PMN technology must allow implementation of systems that comply with life-safety / emergency voice-evacuation standards such as EN54-16 and UL 2572.

3.19. **Public Standards Compliance.** In order to protect and stabilize future options for manufacturers and customers, PMN standards must be open and public.

4. STRUCTURE OF THE OCA SPECIFICATION

The OCA Specification has three divisions, as follows:

- **OCF,** the OCA Framework. This is the architectural description of OCA and its mechanisms.
- **OCC**, the OCC Class Structure. This is the hierarchical structure that defines OCA's control and monitoring repertoire.
- **OCP.1, OCP.2, etc.** Definitions of OCA protocols for various communications media.

5. OCA DESIGN FEATURES

5.1. Protocol Independence

5.1.1. Control Protocol Independence

OCA is called an "architecture" rather than a "protocol" because it is a framework for designing a family of control and monitoring protocols. Depending on communication medium, the actual protocol used may vary.

OCA protocols are identified by codes of the form **OCP.1, ... OCP.n**, where each code designates a particular protocol. At this time, the following protocols have been identified:

- **OCP.1** OCA protocol for TCP/IP networks
- **OCP.2** OCA protocol for USB connections and other byte-serial environments.
- **OCP.3** Text version (probably JSON or XML)

To date, only OCP.1 has been designed.

5.1.2. Media Transport Protocol Independence

OCA is designed to be compatible with today's media transport protocols. The basic OCA 1.1 Specification has no specific media transport bias. Using normal OCA expansion mechanisms, particular OCA versions may be configured to suit the specific aspects of different media transport schemes.

5.2. Object Orientation

5.2.1. Concept

The OCA specification is expressed in object-oriented design terms. Communicating devices are described as sets of objects. Each object is an instantiation of a specific class, and has a state and a set of procedural actions called “methods” that are defined by that class.

An OCA protocol is defined by:

- The set of OCA class definitions, which defines the types of objects that may exist within devices.
- Protocol-specific naming and addressing rules, which define how the objects and their attributes are identified.
- Protocol data unit (PDU) formats, which specify the actual formats of transmitted and received data.
- Protocol data unit (PDU) exchange rules, which define the communication sequences used to effect information exchange.

5.2.2. Classes

OCA classes are defined as nodes in a tree structure rooted in a single, elemental node (the “root class”), and arranged in order by inheritance. Inheritance means that each class is considered to be a specialized entity derived from another, less specialized class (the “parent class”). A class exhibits ("inherits") all the features of its parent, except where that class's definition specifically overrides an inherited feature.

Every OCA class consists of the following:

1. A set of elements (properties, methods, and events) -- see 5.2.2.1, below.
2. A unique class identifier - for details, see [10].
3. Exactly one parent class, except for the base class, which has no parent.

In OCA, the rules for inheritance are:

1. A class must implement all the properties, methods, and events of its parent class.
2. A class may override (i.e. redefine) one or more of its parent class's methods.
3. A class may define methods of its own that are not in the parent class.
4. Any given class except the base class inherits from exactly one other class.

These particular rules guarantee that new features can be added to the protocol in an upwards-compatible manner, by creating new classes from the old ones. New classes support, at a minimum, the functions and interfaces of their parents.

5.2.2.1. Methods, Properties, and Events

Like classes in programming languages, OCA classes have elements which allow access to their data and operating states. In programming classes, these elements define storage references and procedure calls and callbacks (events). In OCA, however, class elements are used to define protocol exchanges between devices and controllers. Because OCA includes a number of different protocols (OCP.1, OCP.2, and so on), the detailed relationship between class elements and protocol elements varies. Nevertheless, the set of class elements is the same for every OCA protocol.

The elements of classes are as follows:

- A class's control or monitoring data (i.e. the parameters that are visible on the control network) are represented by class Properties.
- The class provides appropriate Get() and Set() Methods that controllers may invoke to retrieve and change property values and class operating states.
- Classes also may define Events, which are callbacks that devices may initiate to inform controllers of specific occurrences. To receive events, controllers must first subscribe to them. See section 7, **Events and Subscriptions**, below.

5.3. Messages

OCA control and monitoring operations are implemented by messages passing between objects. The term "control message" (or just "message" where the context is clear) means an OCP protocol data unit (PDU) sent from one object in a particular device to another object, usually in different devices.

Generally speaking, each message contains a method call, a method return status, or a notification of the occurrence of a significant event.

The specific format of OCA messages depends on which OCA protocol is being used. For example, the format used in TCP/IP networks is documented in [14].

With rare exceptions, OCA messages are fully acknowledged, with informative status indications in the return messages. OCA does not use multicasting for messages, for reliability reasons.

OCA defines a "fast" message type that may be used for noncritical traffic such as level meter updates. Fast messaging is not acknowledged and may be multicast, but cannot be used for regular commands. Fast messaging is used by the Subscription mechanism, described in section 7, **Events and Subscriptions**, below.

5.4. Programming Note

The fact the OCA protocols are described in the language of object-oriented design does not imply that implementations of those protocols must be programmed in an object-oriented style.

The implementable content of each OCA protocol is a set of protocol data unit (PDU) formats and exchange rules. Exchange and processing of those PDUs may be programmed in any style, as long as, in the end, the correct data is exchanged.

6. DEVICE MODEL

The OCA device model is OCA's control model for the device. In other words, it is the device as seen by OCA. The internal implementation structure of the device may or may not resemble its device model; the only requirement is that in protocol exchanges device must *behave* as if it had the structure given by its device model.

Generally speaking, the OCA device model tends to differ from device models in other control architectures [1,5,6,8,9,16,17,19] in several ways:

1. It is not hierarchical.
2. It does not predefine specific processing configurations, signal processing modules, device types, or device families.
3. It does not define controller user interfaces or user interface elements.
4. It has strong support for dynamically reconfigurable devices.
5. By comparison, its repertoire of management functions is relatively rich.

Figure 1 is an overview of the OCA Device Model.

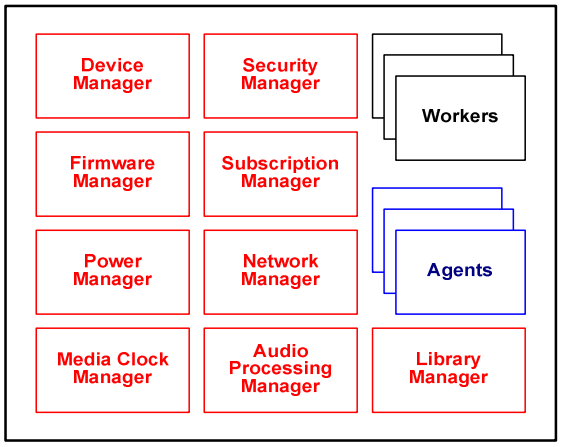

Figure 1. OCA Device Model Overview

Elements of the device model are as follows:

- **Managers.** Manager objects are control objects that affect or report basic attributes and overall states of the device. There is only one instance of each manager class (i.e. one Device Manager, one Security Manager, etc
- **Workers.** Worker objects are control interfaces to the device's application functions - audio mute switches, gain controls, equalizers, level sensors, overload sensors; video camera controls, signal properties, image processors, et cetera.

  Workers are classified into **Actuators**, which control application functions (e.g., a switch); **Sensors**, which detect and report signal parameters and other values back to controllers (e.g., a signal level sensor); **Blocks** and their associated **Block Factories**, which allow the grouping of related objects into collections; and **Matrices**, which allow objects to be assembled into addressable two-dimensional arrays.
- **Agents.** Agent objects are intermediaries that affect control actions within the device. An agent does not itself represent a signal processing function, but instead changes a signal processing parameter in one or more associated workers. For example, there is an agent class named OcaGrouper which implements complex grouping of control parameters in a manner that resembles VCA grouping in analogue systems.

6.1. Device Configuration Types

An OCA device is described as **fixed**, **pluggable**, **partially configurable**, or **fully configurable**.

- **Fixed** means that the device has a permanently assigned object repertoire and signal flow topology, defined at time of manufacture.
- **Pluggable** means that the device's object repertoire and/or signal flow topology may be changed while the device is offline, by plugging and unplugging of hardware modules, adjustment of physical controls, reloading or readjustment of software, or other manual means.
- **Partially configurable** means that the device's signal flow topology may be changed while it is online, via OCP commands.
- **Fully configurable** is a superset of partially configurable, with the addition that OCP commands may be used to create and delete objects inside the device.

Fixed and Pluggable devices are sometimes called **static** devices, because controllers can't change their configurations. Partially Configurable and Fully Configurable devices are sometimes called **dynamic** devices, because controllers can change their configurations during operation.

6.2. Object Addressing

Within an OCA device, each object (i.e. each instantiation of a specific class) is identified uniquely by an object number (ONo). Assignment of Onos is accomplished as follows:

- In fixed and partially configurable devices, Onos are assigned at time of manufacture.
- In pluggable devices, Onos are assigned at device setup time.
- In fully configurable devices, Onos are assigned at time of object creation.

In specific implementations, developers may find it useful to use specific object numbering schemes to optimize

device performance and/or ease object management. This is fully permitted (and even encouraged) by OCA.

OCA makes only three rules about ONo values:

1. ONo values 0 through 4096 are reserved by OCA for managers and possible other future objects that require predefined object numbers.

2. No two objects may have the same ONo.

3. No two Onos may refer to the same object.

6.3. Worker classes

In any device, any worker class may be instantiated as many times as required for control of application function

6.3.1. Actuators

Actuators control device signal processing and housekeeping functions. Details are in [11,12,13].

[11] defines 36 Actuator classes. Examples are:

- **OcaGain** controls a gain function.
- **OcaMute** controls a signal mute function.
- **OcaSwitch** controls a multiposition selector.
- **OcaFilterParametric** controls a parametric equalizer section.
- **OcaDelay** controls a signal delay.
- **OcaDynamics** controls a limiter, compressor, expander, or gate.

6.3.1.1. Detailed Example: OcaGain

To see how classes are generally constructed, we examine OcaGain in more detail. OcaGain is an Actuator, but its general way of working is the same for all classes, be they Workers, Managers, or Agents.

6.3.1.1.1. Lineage

OcaGain's class tree lineage is shown in Figure 2. In this case, the topmost three classes - OcaRoot, OcaWorker, and OcaActuator -- are abstract classes that will never be instantiated per se. They are present in the class tree to express certain common characteristics of Objects, Workers, and Actuators, respectively.

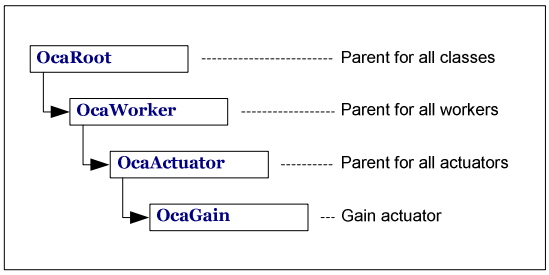

Figure 2. OcaGain's Lineage

6.3.1.1.2. Properties, Methods, and Events

OcaGain's properties, methods, and events are a combination of its own specific elements and elements inherited from its ancestors,. The complete set of properties is given in Table 1.

Name	Description	Defined in
ClassID	Class ID of OcaGain class	OcaGain
Class Version	Version number of OcaGain class	OcaGain
Object Number	Object no. of OcaGain instance	OcaGain
Lockable	True if object can be locked.	OcaRoot
Role	Role of object in device, e.g. "Channel 1 Gain"	OcaRoot
Enabled	True if object is enabled, false if object is disabled	OcaWorker
Ports	Collection of input and output ports that this object has	OcaWorker
Label	Purpose of object in system, e.g. "Elvis Vocal Gain"	OcaWorker
Owner	Object no. of containing block	OcaWorker
Gain	The gain value in dB	OcaGain

Table 1. OcaGain Properties

Properties are accessed via method calls, as documented in [10].

Objects are capable of emitting control signals called "Notifications" to inform controllers of the occurrence of defined types of events. The OCA mechanism for notifications is called the Event and Subscription mechanism - see section 7, **Events and Subscriptions**, below. Specific events and their notifications are documented in [11].

6.3.2. Sensors

Sensors detect the value of some parameter and transmit it back to controllers. Through the use of the OcaNumericObserver agent (see section 6.4.2), a sensor's reading may be transmitted conditionally (e.g. when it exceeds a defined threshold) or periodically.

[11] defines 20 Sensor classes. Examples are:

- **OcaLevelSensor s**enses a signal level
- **OcaAudioLevelSensor s**enses an audio signal level using standard VU or PPM averaging and ballistics.
- **OcaTemperatureSensor s**enses a temperature.

6.3.3. Blocks

A *block* is a special kind of worker that is able to contain other objects, including workers, agents, and other blocks. As well, it can define the signal flow topology among the workers. Managers, however, do not belong to blocks.

We refer to an object inside a block as a *member* of that block. We refer to the block which contains an object as the object's *container*. Each block is described by a class (the *block class*). The base class for all block classes is named OcaBlock.

A block class does not represent specific audio processing in the way that other worker classes do. The block class simply represents a collection of workers, agents, and nested block, and the signal flows within that collection.

Every OCA device has one block called the Root Block that contains all of the device's workers, agents, and, if any, blocks. Advanced devices may contain nested blocks, to any level. Fully reconfigurable devices may allow the creation, modification, and deletion of blocks.

6.3.3.1. Block Contents

6.3.3.1.1. Object Set

Every worker and every agent is a member of exactly one block. In simple devices, they all belong to the Root Block.

The *object set* of a block is the set of workers and agents it contains. Thus, the object set of the Root Block contains all the workers and agents inside the entire device. Some of those workers and agents might be inside nested blocks.

The OCA representation of an object set is called an *object list*. OcaBlock offers methods which allow retrieval of the object list of the block in question. These methods allow retrieval only of the directly contained objects, or full recursive retrieval of all directly contained objects plus all objects inside sub-blocks, sub-sub-blocks, et cetera.

For fully reconfigurable devices, OCA offers methods that allow controllers to modify a block's object set, and to create and destroy whole blocks.

6.3.3.1.2. Signal Flow

Most blocks will contain Worker objects. Workers are signal processors and monitors that send and receive signals. To describe the flows of these signals, the block uses mechanisms named *port, block port, signal path,* and *signal flow.*

- **Ports.** For the purpose of defining media (i.e. audio/video) signal flow, every worker has one or more *signal ports* (or just *ports*). A port is an *input port* if it represents a signal flow into the processing function the worker represents. A port is an *output port* if it represents a signal flow out of the processing function the worker represents.
- **Block Ports.** Blocks themselves have signal ports, called *block ports*. Block ports are intermediate ports that exist to define connection points on the exterior surface of the block . A *block input port* is a connection point for signals entering the block. To objects inside the block, an input block port appears as an output (signal source) port. A *block output port* is a connection point for signals leaving the block. To objects inside the block, an output block port appears as in input (signal sink) port.
- **Signal Paths.** A *signal path* is an internal connection between an output port and an input port in the same device. An *internal signal path* is a signal path between two objects in the same block or between two blocks. An *external signal path* is a signal path between two objects in disparate blocks (OCA allows signal paths to cross block boundaries without going through block ports). A signal path's input and output ports will usually be on different objects, but OCA does not require this.
- **Signal Flow.** The set of all signal paths with at least one end inside a block is called the block's *signal flow*. The block's signal flow does not include signal paths that extend from block ports to other ports outside the block. Such paths belong to the containing block(s). The OCA representation of a signal flow is called a *signal flow list*.

Media transport connections between one device and another are not considered part of device signal flow.

OcaBlock offers methods which allow retrieval of the signal flow list for the block in question. These methods allow either retrieval only of the directly contained signal paths, or full recursive retrieval of all directly contained

signal paths and all signal paths inside sub-blocks, sub-sub-blocks, et cetera.

Figure 3 shows a diagram of a signal flow involving four blocks. The illustration uses small circles for worker ports, large circles for block ports, and simple lines for signal paths. Blocks are shown with rounded corners, other objects with square corners.

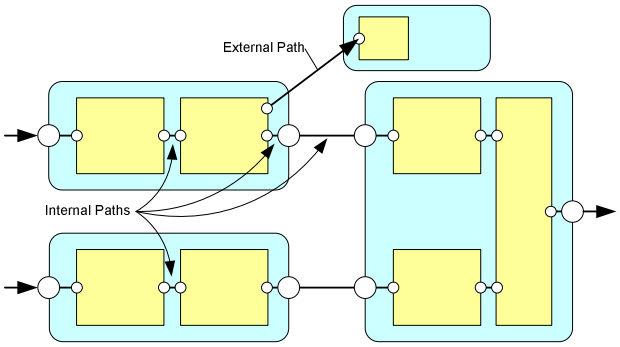

Figure 3. Signal Flow Diagram

6.3.3.2. Abstract Nature of Blocks and Object Numbers

OcaBlock is an abstract class which makes minimal assumptions about device structure and control flow.

There are no predefined block types. Devices may have any arrangement of blocks or no blocks other than the Root Block. OCA makes no assumptions about how blocks are populated, nested, or interconnected. Blocks may be nested to any level.

There is no hierarchical object addressing scheme based on block containment. Object numbers (see section 6.2) are simple integers whose values are not semantically significant to OCA.

Blocks do not aggregate control functions -- i.e., they do not provide grouping or mastering. Such aggregation is provided by the OcaGrouper class. The OcaGrouper mechanism is independent of block boundaries, and fully supports multiple overlapping grouping functions -- see section 6.4.3.

6.3.3.3. Blocks, Controllers, and Interoperability

OCA intentionally does not define standard block types. With the ever-expanding range of options available to designers of DSP-based products, OCA's designers did not consider it feasible for OCA to represent the wide and growing range of device architectures in the pro audio industry. However, within subsections of the industry, defining standard block types may be practical. Where that is the case, the definitions of such blocks may be issued as best-practices documents by the AES, by the OCA Alliance, or by others.

It is expected that OCA will mainly be used in applications where controllers are specifically configured for the products and systems at hand. Those controllers will normally be aware of the specific block structures of the devices they control. The advantage of OCA in such cases is that the controllers may be configured using elements which do not have to differ from manufacturer to manufacturer.

ON the other hand, OCA does provide straightforward and efficient methods for a controller to discover the complete block structure of an OCA device, including all objects and all signal paths contained in every block, and the nesting structure of all blocks Therefore, a sufficiently clever controller could generate a canonical user interface automatically for any OCA device. However, the automatic generation of *commercially acceptable* user interfaces for devices, and, even more, for whole systems is an extremely difficult problem, and one that exceeds the claimed scope of OCA.

6.3.3.4. Examples

Here are a few block examples. In the diagrams, circles represent ports.

6.3.3.4.1. Simple Mic Channel

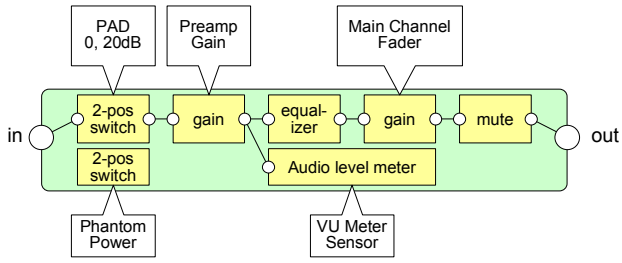

Figure 4. Simple mic channel

This example illustrates three OCA principles:

- Objects of the same class (e.g. two-position switch, gain) may be used for different purposes within a device.
- Not all objects (e.g. phantom power switch) need have signal connections.
- Sensors (e.g. audio level meter) do not have signal outputs.

To aid in managing objects, OCA supports the storage of two text strings inside each worker, as follows:

- The text property Role is a read-only string that gives the role of the worker in the device; e.g. "Preamp Gain".
- The text property Label is a read/write string that allows controllers to designate the function of the worker in the application context, e.g. "Elvis Vocal Gain".

6.3.3.4.2. Two-Channel Mic Mixer

This example illustrates the use of blocks in larger assemblies. The mic channel block of Figure 4 is replicated and combined with another gain control to make a simple mic mixer.

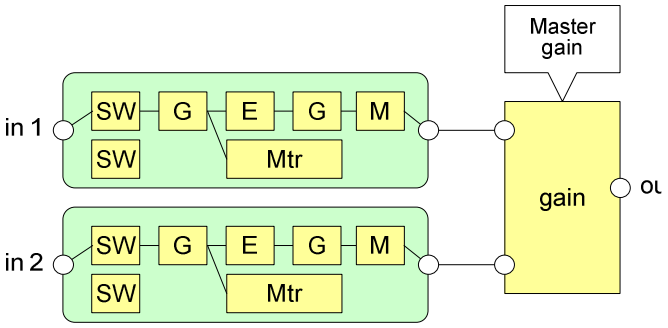

Figure 5. Two-channel Mic Mixer

In this case, the master gain object has been given two input ports in order to represent the mixing function.

Note that there is no explicit object for mix bus summing amplifier. The reason for this is another OCA principle:

- OCA only models the control and monitoring functions of a device, not its entire function. If a device function has no remote control features, OCA does not model it.

For this simple mixer, the summing amplifier does not have parameters that are remotely controllable. Therefore, it has no corresponding OCA object.

More sophisticated mixers might have control and/or monitoring functions associated with the summing subsystem. In that case, the OCA signal flow would include one or more objects to represent those functions.

6.3.3.4.3. Mic Channel Using Nested Blocks

We will now consider the equalizer object shown in the mic channel model in Figure 4.

A basic mic channel equalizer might contain a high-pass filter section, followed by perhaps three parametric equalizer sections.

OCA defines a class that can be used to model a high-pass filter (class OcaFilterClassical), and another class that can be used to model a parametric equalizer section (class OcaFilterParametric). Thus, to model the mic channel equalizer, we will use one OcaFilterClassical object, followed by three OcaFilterParametric objects.

We can just add these to the mic channel inline, as shown in Figure 6.

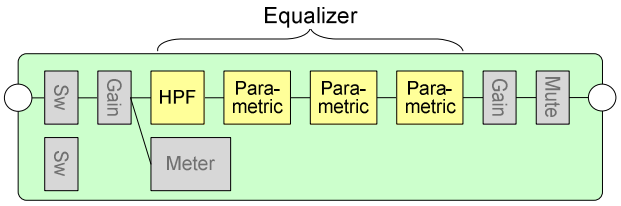

Figure 6. Mic channel with EQ sections inline

In this figure, the object labeled HPF is an object of class OcaFilterClassical, and the objects labeled Parametric are objects of class OcaFilterParametric.

Alternatively, we could define a block named, say, MicEqualizer and nest it inside the mic channel block.

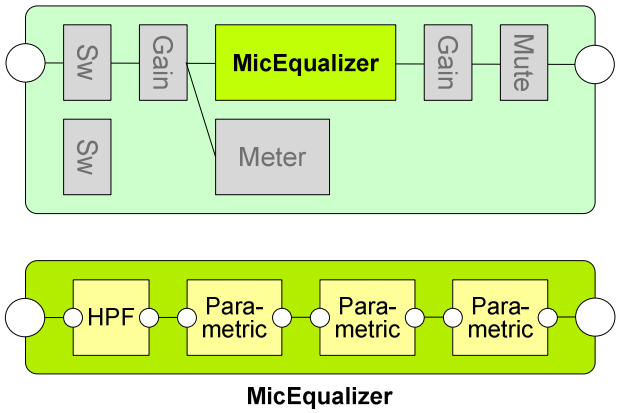

Figure 7. Mic channel with MicEqualizer block

This construct is tidier and will be easier to use in reconfigurable devices. Also, it might make controller implementation more straightforward, particularly if the same equalizer were used in various parts of the device, and/or in other products.

6.3.3.5. Block Factories

In fully reconfigurable devices, controllers must be able to build blocks, populate them with objects, and "wire" signal paths among those objects. OCA provides two ways of doing this.

1. The controller can instantiate an object of class OcaBlock, then send commands to the device which create and interconnect objects inside the new block. OR

2. The controller can use a *block factory*.

A block factory is a worker whose job is to construct fully populated and "wired" blocks. Each block factory is an instance of class OcaBlockFactory that has been configured to construct one specific kind of block, with a predefined set of objects and signal paths. Block factories are useful when controllers must create multiple blocks, all of the same kind.

A block factory is considered to be a container of prototype objects and prototype signal flows which are realized each time the factory constructs a block.

There are two ways of creating block factories. Either or both of these methods may be implemented, depending on device type.

1. One or more block factories may be defined at time of manufacture or, for pluggable devices, at time of device setup. In this case, these block factories provide controllers with a predefined repertoire of block types which they may instantiate.

2. The controller may create block factories using specific OCA commands to define the configurations of blocks they will create. In this case, controllers are free to define new block types which may be subsequently instantiated.

6.3.4. Matrices

The OCA matrix is a generalization of the familiar audio matrix concept. OCA matrices are rectangular arrays of identical worker objects that share one or more common input and output busses for the rows and columns, respectively.

Figure 8 shows two typical OCA matrices.

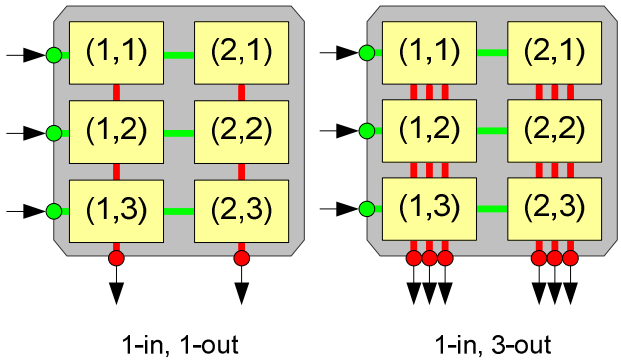

Figure 8. Typical OCA matrices

If the workers are switches, the matrix would represent a conventional switching matrix; if the workers are gain controls, the matrix would represent a conventional mixing matrix. However, in OCA the workers can be of any class, even blocks. For example, we could construct a matrix of blocks such as shown in Figure 9 and Figure 10.

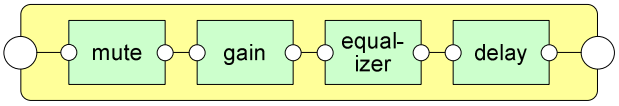

Figure 9. A complex matrix element

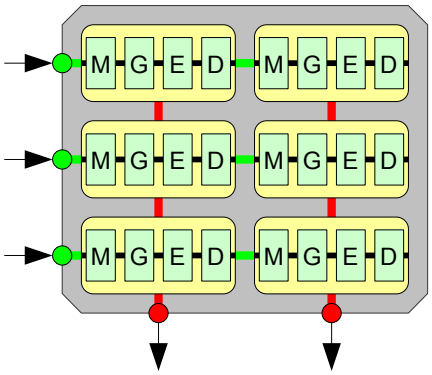

Figure 10. Matrix of complex elements

6.3.4.1. Matrix Addressing

As suggested by Figure 8, OCA matrix elements are addressed by coordinates. The OCA convention is to use (x, y) coordinates, where x is the horizontal (column) coordinate, and y is the vertical (row) coordinate.

Normal coordinate values range from 1 to the number of rows or columns.

The special value zero is used to denote the entire row or column. So, for example, (0,2) means the entire second row.

6.3.4.2. Matrix Structure

An OCA matrix is based around an object which is an instance of the OcaMatrix class, but that instance does not include the objects which constitute the matrix elements. Those are instantiated on their own, and are called the members of the matrix. Their class is called the matrix's member class. All of a matrix's members must be of the same class.

The members of an OCA matrix must reside in the same device as the matrix instance.

In addition to the matrix object and the members, the OCA matrix includes one additional instance of the member class, called the matrix proxy. The matrix proxy is used by controllers to access the settings of matrix members via coordinates.

A complete matrix is shown in Figure 11.

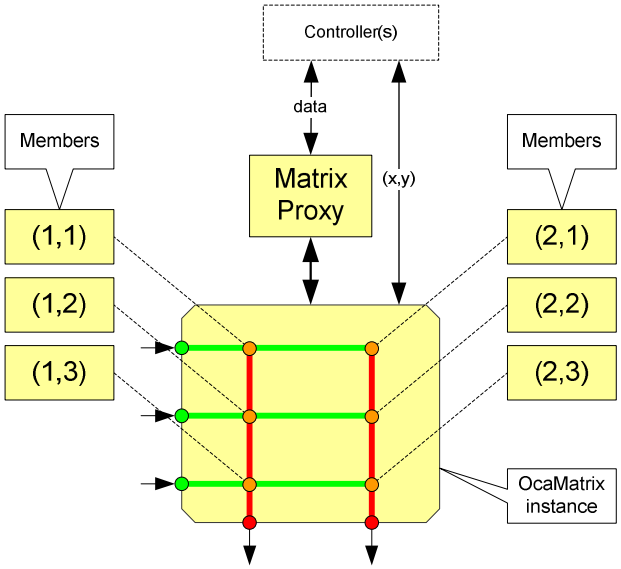

Figure 11. OCA Matrix Structure

6.3.4.3. Applications of Matrices

Because they are defined rather generally, OCA matrices may find use not only for the representation of traditional audio matrixing facilities, but for other applications which require addressing collections of objects as one- or two-dimensional arrays. Such applications may or may not make use of matrix input and matrix output ports.

For example, a multichannel loudspeaker crossover might be represented as a matrix of crossover channels, with each channel being a block containing the usual filters, gain elements, delays, and dynamics controls. In this case, the channels might share a common input bus which could be represented by a matrix row port. However, they would have separate output ports, so no matrix output ports would be defined.

6.4. Agents

OCA 1.1 defines seven agent classes. Each of these implements a notable OCA mechanism, as described in the following sections.

6.4.1. OcaRamper

The **OcaRamper** class defines a mechanism (ramper) by which controllers can cause devices to execute incremental and/or prescheduled parameter changes. The rationale for rampers is:

- It is usually impractical to implement incremental changes (fade-ins, fade-outs, and crossfades) by sending sequences of OCA commands over the network, because the amount of control traffic required would be excessive.
- It is often not possible to implement prescheduled parameter changes in controllers, because application systems may need to execute such changes when controllers are not running.
- In general, timing accuracy of ramping actions will be more precise if not subject to network delays.

Each ramper is bound to a particular worker property. Ramper principal parameters include target property value, ramp duration, ramp start time, and ramping interpolation law.

Each ramper maintains a state property whose value can be monitored by controllers to determine ramping status - waiting to start, in process, complete, aborted, et cetera.

6.4.2. OcaNumericObserver

The **OcaNumericObserver** class defines a property value "watcher" object that monitors the value of a specified numeric property in another object and, under certain conditions, notifies controllers of the value. It is part of the OCA Event and Subscription mechanism, and will be described in section 7.

6.4.3. OcaGrouper

The class OcaGrouper defines objects ("groupers") responsible for aggregating property values. By "aggregating", we mean associating property values in such a way as to make them controllable as a single value. Ag-

gregation supports audio control functions variously known as ganging, linking, mastering, submastering, and VCA mastering.

The theory of grouping is discussed in [7].

No signals pass through groupers - they affect control flow only. In this respect, they resemble traditional VCA masters more than traditional mix groups and subgroups.

An *actuator grouper* allows control of many actuator objects from a single input value; a *sensor grouper* allows observing many sensor objects via a single output value. Actuator groupers are described below; the details of sensor groupers are not yet developed.

OCA groups aggregate whole objects, not individual properties, but still maintain the distinction between different property types. For example, the OcaFilterParametric class represents a parametric equalizer with several properties - frequency, Q, and passband gain. When a grouper groups a set of OcaFilterParametric objects, all of these properties are grouped, but the group parameters are kept separate from each other. Thus, the grouper would provide separate group parameters for each of frequency, Q, and passband gain.

The term *group setpoint* refers to a group's setting for a particular property.

6.4.3.1. Multiple Group Membership

In a working media system, many objects will be members of multiple groups. For example, in a multiway stereo sound reinforcement system, the left channel woofer power amplifier's gain control object might be controlled by a master gain group, a left-side gain group, and a woofer gain group.

For a property that belongs to an object which is a member of two or more groups, its setpoint will depend on the cumulative effect of the corresponding group setpoints of all the groups of which it is a member. Because all properties have range limits, it is possible that the cumulative effect of all applicable groups might drive a property out of range. This effect must be prevented or at least managed. Therefore, the grouper must know about all the groups and which objects belong to which, and must have algorithms which use this knowledge to forestall range limit issues.

6.4.3.2. Group Structure

As noted above, the grouper must know about all the groups and all of the objects which belong to them. We use the following terminology:

- **Citizen** An object of which a grouper is aware.
- **Group** A group of which a grouper is aware.
- **Enrollment** The binding of a citizen to a group.
- **Member** A citizen that is enrolled in a group.

It will be helpful to visualize a grouper as a matrix whose rows are groups and columns are citizens, and where each cell contains information relating to the membership of the citizen (column) in the group (row).

A grouper holds references to its citizens, but does not create, destroy, or contain them.

A grouper's citizens may reside anywhere on the network -- they do not need to be instantiated in the same device as the grouper.

For each grouper, all the citizens must be instances of the same class.

Each group is accessed via a proxy object, called the *group proxy*. The class of a group proxy is identical to the class of the group's members. To change a group setpoint, a controller changes the corresponding property of the group proxy. Group proxies are automatically instantiated and deleted by the grouper, and always reside in the same device as the grouper. A grouper that manages (N) groups will have (N) group proxies.

A grouper does the following:

- Creates / deletes all the groups and group proxies.
- Registers / de-registers all the citizens.
- Enrolls / de-enrolls citizens in / from groups.
- Computes and sets new property values when group setpoint values change.
- Manages overrange and underrange conditions proactively, so that citizens are not asked to assign out of range values to their properties.
- Handles error conditions that arise if / when connections between grouper and citizen are lost.

Figure 12 shows a typical grouper configuration for controlling a stereophonic set of three-way loudspeaker systems.

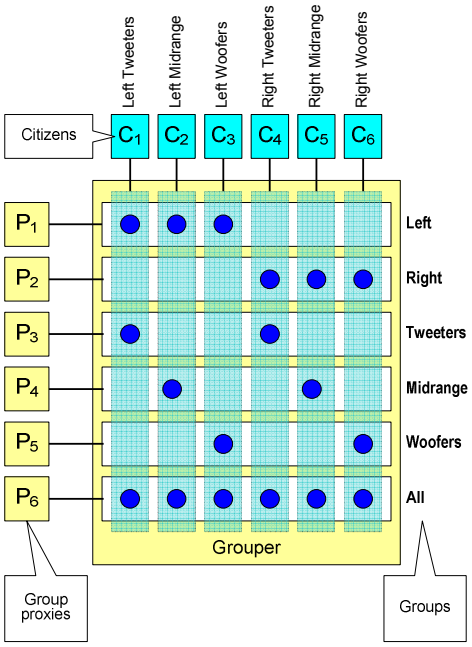

Figure 12. Typical Grouper.
Squares are objects, circles are enrollments.

6.4.3.3. Aggregation and Saturation Rules

Citizens have aggregation rules, which determine the algorithms by which their setpoint values are computed. Groups have saturation rules, which control handling of overrange conditions. These rules will vary from citizen class to citizen class, property type to property type, and product to product.

The selection of aggregation and saturation rules is complex, as is the definition of appropriate connection-loss behavior. It is expected that manufacturers will create subclasses of **OcaGrouper** to implement their own choices.

6.4.3.4. Grouper Deployment

Groupers may be instantiated in ordinary devices, or in dedicated OCA control servers, or in system controllers. The choice among these is a system design decision. OCA will support a flexible range of deployment scenarios.

6.4.4. OcaLibrary

The **OcaLibrary** class defines mechanisms for the handling of parameter value sets that are pre-stored in devices. Such parameter sets have historically been called "presets", "patches", "memories", "cues", and probably other things.

OCA's features in this area are organized around the Block mechanism described in section 6.3.3.

The library feature is optional. OCA devices need not implement any library-related objects or functions.

OCA defines two kinds of prestored parameter value sets:

6.4.4.1. ParamSet

A *ParamSet* is a prestored set of property values for one particular class of block.

Storing a ParamSet's values into the designated objects in an instance of a block is called *applying the ParamSet* to the block.

A ParamSet need not contain values for all of the properties in the target block. Indeed, it might contain as little as one value.

Applying one ParamSet to a block does not necessarily erase the effect of previous ParamSets. If one ParamSet affects different properties from a previously applied one, then both will continue in effect.

The block class for which a ParamSet is defined is called the ParamSet's *target block class*.

A single ParamSet may be applied to any number of blocks, as long as all of the blocks are instances of the ParamSet's target block class.

Read-only properties (e.g. ClassID) will not be saved or restored by this mechanism.

ParamSets are defined for block classes, not for specific blocks (instances). For example:, suppose a mixing console defined a block class named *InpChannel* to represent one input channel. A 32-input console would have 32 instances of this class. If the console defined a ParamSet for InpChannel, that ParamSets could be applied to any instance of InpChannel, i.e. to any of the 32 channels.

The association of a particular ParamSet P with a particular block B is called an *Assignment of P to B.* Such an assignment represents a potential future application of P to B. When P is subsequently applied to B, we say that the assignment is *executed.* These terms are used for patches, defined next.

6.4.4.2. Patch

A *Patch* is a collection of Assignments. The act of executing all the Assignments in a Patch is called *applying the Patch* to the device.

6.4.4.3. OcaLibrary Structure

A collection of ParamSets is called a *ParamSet Library*. A collection of Patches is called a *Patch Library*. OCA does not restrict the number of ParamSet Libraries or Patch Libraries that a device may implement.

Each library in a device is represented by an **OcaLibrary** instance. All of these instances, both for ParamSet and Patch Libraries, are anchored in the **OcaLibraryManager** object.

The binary format of a ParamSet is device-dependent and is not defined by OCA. Within the OCA protocol, a ParamSet is treated as an undifferentiated binary large object (BLOB). OCA includes primitives for upload, download, creation, and management of ParamSets, but these primitives do not include functions that "look inside" those ParamSets.

6.4.5. OcaNetwork

In OCA, the concept of network is generalized: A network may support control and monitoring, or media transport, or both. For networks that provide media transport, the control interface is generic, in order to allow OCA to work with media networks of various types.

OCA supports devices that belong to more than one network at a time. The multiple networks may all be of the same type, or they may be of assorted types.

In what follows, a network that supports control and monitoring will be called a "control network". A network that supports media transport will be called a "media transport network"; a network that supports both will be called a "full media network".

6.4.5.1. Network Classes and Objects

OCA network support is anchored in the Network Manager (class OcaNetworkManager). The Network Manager links to all the Network objects in the device.

Most devices will have one Network object for media transport and one Network object for control and monitoring. In some cases, these two Network objects may be combined into a single one. More complex devices may have multiple Network objects for media transport and/or control.

The root class for Network objects is OcaNetwork. The specific implementation of this class will depend on what kind of network is to be represented.

Some implementations may need to create more specific subclasses of OcaNetwork in order to represent their network types adequately. The definitions of appropriate OcaNetwork subclasses are a part of the definition of each OCP.n protocol. For example, OcaNetwork subclasses for TCP/IP networks are documented in [14].

6.4.5.2. General Features

Each OcaNetwork instance provides the following features for the network it represents:

- Functions to start up, pause, and shut down the interface for the network.
- Standardized network status indication.
- Ability to get and set the host name or ID by which the device advertises itself on the network.
- Identification of the hardware link type (TCP/IP Ethernet, USB, etc.) of the network.
- Identification of the system interface the network uses for input and output.
- Identification of the control (if any) and media transport (if any) protocols the network uses.
- Transmission performance and error statistics.

6.4.5.3. Media Transport Management

OCA defines an interface for the management of media transport connections. Through this interface, OCA controllers can access media network connection management functions to set up, monitor, and tear down media transport connections between devices.

OCA media transport management is based on two elements, as follows (see Figure 13):

- **Media Ports**. A media port is an object that is capable of being bound to a set of internal signal paths and to an external media transport endpoint. Media streams travel from external endpoints through media ports to the objects inside the device, and vice versa.

- **Media Connections**. A media connection is a data element that is part of a Media Port. It describes the binding of the media port to an external media transport endpoint for the purpose of sending or receiving media stream data.

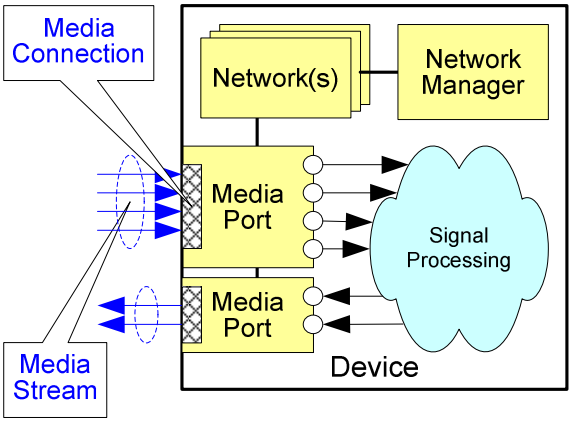

Figure 13. Media networking control elements.

The Network object includes a feature for identifying a media transport connection as a secure one. However, the actual implementation of media transport security is left up to the specific network implementation.

OCA places no restrictions on media stream formats, sampling rates, encodings, et cetera.

6.4.6. OcaMediaClock

The **OcaMediaClock** class describes a particular internal or external media clock which the device uses. It includes features for specifying clock source, rates, lock states, and so on. OCA allows a device to define any number of media clocks, each one represented by its own instance of OcaMediaClock. .

The media clock feature is optional. If an OCA device has no network-controllable clocking features, it need not instantiate OcaMediaClockManager or OcaMediaClock.

6.4.7. OcaEventHandler

OcaEventHandler is a special class used in the processing of OCA events. It is described in section 7.

6.5. Manager Classes

Table 2 shows all of the OCA manager classes. The Required or Optional column shows which managers are required for all OCA-compliant devices, and which are optional. For required managers, not all of the parameters need be implemented for OCA compliance. Details are in [15].

As noted above, each manager class is instantiated at most once per device, and with a well-known object number.

Obj No	Class & Description	Required or Optional
1	**OcaDeviceManager** Manages information relevant to the whole device - model and serial number, device name and role, overall operating state, device update lock, etc.	Required
2	**OcaSecurityManager** Manages security keys.	Required
3	**OcaFirmwareManager** Performs firmware updating.	Required
4	**OcaSubscriptionManager** Manages subscriptions, the constructs by which devices inform controllers of significant events.	Required
5	**OcaPowerManager** Manages device power state, including multiple power supplies, battery supplies, and so on.	Required
6	**OcaNetworkManager** Manages the control and media transport network(s) to which the device is connected.	Required
7	**OcaMediaClockManager** Manages media clock selection and control.	Optional
8	**OcaLibraryManager** Manages device patches and presets, the OCA elements that handle prestored device configurations and parameter settings. See section 6.4.4.	Optional
9	**OcaAudioProcessingManager** Manages global audio-related parameters such as headroom and sampling rate.	Optional

Table 2. OCA Managers

7. EVENTS AND SUBSCRIPTIONS

In the control and monitoring of signal processing and routing, it is often necessary for devices to report the values of varying parameters automatically and on a continuous basis. OCA provides for this function via the subscription mechanism, in which certain objects can be made to transmit property value update messages to other objects automatically and repetitively.

Objects that support subscriptions are called *emitters*, and the messages they emit to announce events are called *notifications*.

OCA includes specific features to address three particular issues in the processing of subscriptions:

1. **Efficiency**. OCA defines two types of subscriptions: *reliable*, in which notifications are transmitted via the normal OCP command communication mechanism; and *fast*, in which notifications are transmitted via less reliable but more efficient means. For example, in OCP.1 *reliable* notifications are sent via TCP, while *fast* notifications are sent via UDP unicast or multicast.

2. **Multiple subscriptions per emitter**. Emitters can support multiple simultaneous subscriptions. In theory, there is no limit to the number of such subscriptions.

3. **Abandoned subscriptions**. An abandoned subscription is a subscription whose subscriber has crashed or disappeared. When an OCA device detects a subscriber failure, it will delete all subscriptions which were made by the failed subscriber.

Subscriptions are created by the Subscription Manager in response to controller requests. Subscriptions persist until the controller deletes them or until they are abandoned.

A subscription is a binding between two elements, an *event* and an *event handler*.

- An *event* is a specific event type of a specific object. For example, if object number 227 defines event types Start and Stop, then {227,Start} is an event, and {227,Stop} is another event.
- An *event handler* is an **OnEvent** method of an instance of the class **OcaEventHandler**.
- The object that owns the event handler is called the *subscriber*.

The relationship between events and event handlers is many-to-many: an event may have any number of subscribers, and an event handler may participate in any number of subscriptions.

When a subscribed event occurs, the subscription mechanism transmits a notification to the event handler. This notification is in effect a call to the event handling method -- i.e., to the OnEvent method of the subscriber.

7.1. The PropertyChanged Event

OCA does not have a large repertoire of different event types. The small repertoire is sufficient mainly because the root class OcaRoot defines an event named PropertyChanged which, when used, fires whenever any of its object's property values changes. Through the class inheritance mechanism, PropertyChanged is defined for every OCA class.

PropertyChanged subscriptions can be used for such purposes as:

- Monitoring signal-induced object state changes;
- Monitoring overall device state;
- For devices with front panels, synchronizing controllers with user adjustments to front-panel controls;
- In multicontroller systems, synchronizing parameter changes among the various controllers.

7.2. Use of Numeric Observers

Sometimes it is efficient for devices to provide a tiny intelligence for monitoring parameter values. There are three common cases for this:

- When monitoring a numeric or logical parameter, the controller wants notification only whenever the property value meets a certain simple numeric test -- for example, when it exceeds a defined threshold.; or
- A controller desires a regular periodic readout of a numeric parameter, regardless of its value; or
- The combination of the above: a controller desires regular periodic readouts, but only when the parameter value meets a certain value criterion.

OcaNumericObserver objects ("Numeric Observers") provide these capabilities - see section 6.4.2.

Each Numeric Observer watches a specified parameter value, and emits an event named Observation whenever its criteria are met.

8. CONCURRENCY CONTROL

OCA supports the use of multiple simultaneous controllers with overlapping control domains, -- i.e., applications in which a particular object may receive directives from more than one controller.

To prevent race conditions in this case, OCA devices support single-threading via a simple object locking mechanism.

OCA does not include a full lock management function. There is no deadlock detection, there are no non-exclusive locks, there are no multi-level locking schemes, et cetera. These functions, if needed, are left up to the controller application.

9. SECURITY

OCA control security is a protocol-dependent element. In OCP.1 networks, security of the control data is handled by using the TCP/IP family's Transport Layer Security (TLS) protocol. Details are in [14].

OCA Security data can be turned on or off globally. Networks with a mix of secure and insecure devices are not supported.

10. RELIABILITY

"Reliability" means the cumulative effect of Availability and Robustness. Fully implemented OCA networks are capable of operating at levels of reliability sufficient to satisfy:

1. international regulations governing emergency evacuation systems.
2. commercial and government uptime and robustness requirements.
3. public media and live performance uptime and robustness requirements.

10.1. Availability

High availability of OCA networks is supported by:

1. Continuous device supervision, via periodic "keep-alive" messages defined as part of OCP protocols.

2. OCP features for efficient network reinitialization following errors and configuration changes.

10.2. Robustness

To support robust implementations, OCA includes mechanisms for operation confirmation, and has fault-tolerant mechanisms that are resistant to PDU losses and device failures.

OCP protocols may use network type specific robustness mechanisms. For example, OCP.1 can take advantage of spanning-tree protocols to increase resistance to network link failures.

10.3. Session Orientation

The general reliability of OCA is supported by its session-oriented design.

1. OCA defines exclusively request/response protocols.
2. Prompt discovery of device failure is supported.
3. Devices must be able to report changing parameters (e.g. signal level) to subscribing controllers.
4. Connection relationships behave appropriately when transport interruptions occur.

11. DEVICE RESET

For recovery from terminal errors, OCA supports the *device reset* operation. A device reset has the effect of returning the affected device to the state it was in immediately following power-up. A device reset causes all of the device's initialization data (e.g. routing information) to be deleted.

A device reset is invoked by the device's receipt of a *device reset message*. In OCP.1, a device reset message is a special message with a unique port number and a 128-bit key (the "reset key"). The value of this key must match a previously-stored value in the device. If it does not, the device reset message has no effect.

Reset key values are set by a particular OCA command, and the memory of key values does not survive a power-up reset. If a device has not received a reset key value since the most recent power-up reset, it will ignore device reset messages. Thus, since it causes a power-up reset, a device reset message will cause the affected device to forget its reset key.

Device resets may be sent via unicast or, where supported by the control network in use, multicast.

12. FIRMWARE/SOFTWARE UPGRADE

For devices with sufficient processing capacity, OCA supports reliable firmware/software upgrading over the network.

OCA requires that network upgrading be implemented in a way which ensures the device can recover from a failed firmware installation. This implies that the device must have a protected bootup downloader that continues to function even when the device's program memory contains an invalid image, or no image.

Security of the firmware/software upgrade is handled through the normal security mechanism for control data (see section 9), since the firmware/software upgrade is triggered via OCP.

13. CONCLUSION

OCA provides an architectural framework for device control in professional media networks. It supports multivendor interoperability, now and in the future. It is amenable to proprietary functionality, and scalable over the entire range of professional media network applications. A proposed open royalty-free public standard, OCA is poised side-by-side with emerging media transport standards to form a complete basis for interoperable professional media networking in the near future.

14. REFERENCES

[1] American National Standards Institute. "E1-17: Architecture for Control Networks" . *Definition of ACN.*

Package of 17 documents plus supporting files. Online:http://webstore.ansi.org

[2] AES SC-10. (1999) , "AES24-1-1999 (w2004): AES standard for sound system control - Application protocol for controlling and monitoring audio devices via digital data networks - Part 1: Principles, formats, and basic procedures (Revision of AES24-1-1995)". *AES24 architecture.*

[3] AES SC-10. (1999) , "AES24-2-tu (w2004): PROPOSED DRAFT AES standard for sound system control - application protocol for controlling and monitoring audio devices via digital data networks - Part 2, data types, constants, and class structure (for Trial Use)". *AES24 control repertoire as defined by its control class structure.*

[4] Jeffrey Berryman, "Technical Criteria for Professional Media Networks," in *Proceedings of AES 44th Conference on Networking* , San Diego , 2011 . *Outlines the range of usecases that OCA is intended to serve.*

[5] Richard Foss and Andrew Eales, "Towards a Standard Model for Networked Audio Devices," in *Proceedings of the AES 44th International Conference - Audio Networking* , San Diego , 2011 . *Includes a helpful overview of current media system control protocols.*

[6] Richard Foss and others, "The XFN Connection Management and Control Protocol," in *Proceedings of the AES 44th International Conference - Audio Networking* , San Diego , 2011 . *XFN is the control protocol being standardized as AES X170.*

[7] Nyasha Chigwamba, Richard Foss, Robby Gurdan, and Bradle Klinkradt, "Parameter Relationships in High-Speed Audio Networks," *JAES* , vol. 60 , no. 3 , pp. 132-146 , March 2012 .

[8] International Electrotechnical Commission, "Common control interface for networked digital audio and video products - Part 1: General," Standard IEC 62379-1, . Online:http://webstore.iec.ch

[9] International Electrotechnical Commission, "Common control interface for networked digital audio and video products – Part 2: Audio," Standard IEC 62379-2, September 2008 . Online:http://webstore.iec.ch

[10] OCA Alliance. (2012, July) , "OCA 1.1 Framework". PDF in OCA 1.1 Specification. *OCA concepts and architecture. The primary basis for this paper.*

[11] OCA Alliance. (2012, July) , "OCA 1.1 Control Classes". PDF in OCA 1.1 Specification. *OCA control repertoire as defined by its control class structure.*

[12] OCA Alliance. (2012, July) , "OCA 1.1. Control Class Diagrams". PDF in OCA 1.1 Specification. *Set of UML (Universal Modeling Language) diagrams illustrating OCA control classes. Companion to 'OCA 1.1 Control Classes'.*

[13] OCA Alliance. (2012, July) , "OCA 1.1 Control Datatypes". PDF in OCA 1.1 Specification. *Control datatypes used by OCA control classes.*

[14] OCA Alliance. (2012, July) , "OCA 1.1 TCPIP Protocol". PDF in OCA 1.1 Specification. *OCP.1, the OCA implementation for TCP/IP networks.*

[15] OCA Alliance. (2012, July) , "OCA 1.1 Minimum Object Configuration". PDF in OCA 1.1 Specification. *Specification of the minimum set of OCA control objects that a device must implement to be OCA-compliant.*

[16] M. Wright. (2002, March) , "The Open Sound Control 1.0 Specification". *Latest complete specification as of this writing. Version 1.1 is in development.* Online:http://opensoundcontrol.org/spec-1_0

[17] A. Freed and A. Schmeder. (2009) "Features and Future of Open Sound Control version 1.1 for NIME" . *Conference paper from NIME 2009 that outlines plans for OSC 1.1. NIME stands for "New Interfaces for Musical Expression".* Online:http://cnmat.berkeley.edu/system/files/attachments/Nime09OSCfinal.pdf

[18] Francis Rumsey, "Audio Networking," *JAES* , vol. 60 , no. 4 , pp. 282-286 , April 2012 .

[19] UPnP Forum. (2008, October) "UPnP Device Architecture 1.1" . *Part of a document set downloadable as a zipfile from the cited URL.* Online:http://upnp.org/resources/upnpresources.zip

Referenced AES publications are available from the AES website, *www.aes.org*.

The referenced **OCA 1.1 Specification** is in the file **OCA 1.1 .zip,** available for free download from the Alliance website Specifications page:

http://www.oca-alliance.org/Technology/Specifications.html